

Module - 3

Array

An array is a collection of elements of the same type placed in a continuous memory locations. Arrays are used to store a set of values of the same type under a single variable.

To read and display 1D array

```
#include <stdio.h>
void main()
{
    int array[5];
    int i;
    for (i=0; i<5; i++)
    {
        printf("\n enter any number");
        scanf("%d", &array[i]);
    }
    for (i=0; i<5; i++)
    {
        printf("%d", array[i]);
    }
}
```

Matrix Addition

```
#include <stdio.h>
void main ()
{
    int A[10][10], B[10][10], C[10][10], i, j, r1, r2, c1, c2;
    printf("enter the number of rows of first matrix");
    scanf("%d", &r1);
    printf("enter the number of columns of first matrix");
    scanf("%d", &c1);
    for (i=0; i<r1; i++)
    {
        for (j=0; j<c1; j++)
        {
            scanf("%d", &A[i][j]);
        }
    }
    printf("enter the number of rows of second array");
    scanf("%d", &r2);
    printf("enter the number of columns of second array");
    scanf("%d", &c2);
    for (i=0; i<r2; i++)
    {
        for (j=0; j<c2; j++)
        {
            scanf("%d", &B[i][j]);
        }
    }
}
```

```

for (i=0; i<4; i++)
{
    for (j=0; j<4; j++)
    {
        C[i][j] = A[i][j] + B[j][j];
    }
}

for (i=0; i<4; i++)
{
    for (j=0; j<4; j++)
    {
        printf ("%d\t", C[i][j]);
    }
    printf ("\n");
}

```

Linear Search

```
#include <stdio.h>
void main()
{
    int i, c, n, a[25], flag;
    printf("enter the number of limit of array");
    scanf("%d", &n);
    printf("enter the array elements");
    for(i=0; i<=n; i++)
    {
        scanf scanf("%d", &a[i]);
    }
    printf("enter the element");
    scanf("%d", &c);
    for(i=0; i<=n; i++)
    {
        if(a[i]==c)
        {
            flag++;
            break;
        }
    }
    if(flag==0)
        printf("the element is not found");
    else
        printf("the element is found : %d", i);
}
```

bubble Sort

```
#include <stdio.h>
void main()
{
    int i, a[100], k, n, temp;
    printf("enter the number of element n");
    scanf("%d", &n);
    printf("enter the elements");
    for (i=0; i<n; i++)
    {
        scanf("%d", &a[i]);
    }
    for (i=0; i<n; i++)
    {
        for (k=0; k<n-i-1; k++)
        {
            if (a[k]>a[k+1])
            {
                temp=a[k];
                a[k]=a[k+1];
                a[k+1]=temp;
            }
        }
    }
    printf("ascending order");
    for (i=0; i<n; i++)
    {
        printf("%d\n", a[i]);
    }
}
```

Transpose of matrix

```
#include <stdio.h>
void main()
{
    int i, j, A[10][10], r, c;
    printf("enter the number of rows and columns of matrix");
    scanf("%d %d", &r, &c);
    printf("enter the elements");
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            scanf("%d", &A[i][j]);
        }
    }
    printf("Entered matrix:\n");
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            printf("%d\t", A[i][j]);
        }
    }
    printf("Transpose of matrix:\n");
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            printf("%d\t", A[j][i]);
        }
    }
}
```

```

{
    printf("%d", A[5]);
}
printf("%d");
}

```

String Handling Functions

1) strrev() - String Reverses
strrev() reverses the string.

Syntax: strrev(string);

Eg: char str[] = "hello";
strrev(str);
printf("%s", str); //olleh

2) strcat()

String Concatenate.

- Strcat Concatenates Second String with First String.
- This function can take two Parameters after executing, value of Second Parameter will be Concatenated with First String.

Syntax: strcat(string 1, string 2);

eg: char s1[100] = "good";

s2[100] = "morning";

strcat(s1, s2);

printf("%s", s1);

printf("%s", s2);

3) Stricmp()

String Comparing Ignoring Case.

Stricmp Comparing two Strings Ignoring Case Sensitivity and return 0, less than 0 and greater than 0 based on strings.

Syntax : Stricmp (String 1, String 2);

4) Strcmp()

Compare two Strings.

Compares two Strings and return "0", If two strings are equal, a negative value if the first is less than the second, a positive value if the first is greater than second.

Syntax : Strcmp (String 1, String 2);

5) Strcpy()

Copy the Content of one String to Other.

Copies one String to another String, there will be two parameters, Second Parameter value will be copied into First Parameter variable.

Syntax: strcpy (string 1, string 2);

eg: char s1[100] = "good", s2[100] = "morning";

strcpy(s1, s2);

printf("%s", s1);

printf("%s", s2);

strcpy copies the contents of the source string (s2) into the destination string (s1). It starts copying from the first character of s2 and continues until it reaches the null terminator '\0'.

Example: strcpy(s1, s2);

(1) strcpy()

strcpy copies the contents of the source string (s2) into the destination string (s1). It starts copying from the first character of s2 and continues until it reaches the null terminator '\0'.

Example: strcpy(s1, s2);

(2) strcpy()

strcpy copies the contents of the source string (s2) into the destination string (s1). It starts copying from the first character of s2 and continues until it reaches the null terminator '\0'.

Module 4

• Advantages of Modular Programming?

1)

- Programs can be designed easily
- Code is short, simple, easy to understand
- Error can easily identified as they are localized to a function
- The same code can be used in many applications
- Allow many programmers to collaborate on the same application.

• Define Formal Parameters and actual Parameters, with an example?

A)

Formal Parameters

- They are the place holders for the values of actual parameters.
- They are the parameters as they appear in function declaration
- They are used in function header
- They are used to receive the values that are passed to the function through function call.

eg: x and y

Actual Parameters

- they contain the values that are passed to a procedure and receive results.
- they are parameters as they appear in function call.
- They are used in function call.
- They may be constant value.
- They are actual values that are passed to the function definition through the function call.

eg.: 2, 3

Q) Differentiate btw Structure and union using eg ?

A) Structure

- The struct keyword is used to define a structure.
- Each variable member occupied a unique memory space.
- Each variable member will be accessed at a time.

union

- The union keyword is used to define union.
- The size of the union is equal to the size of the largest data member.
- Variable members share the memory space of the largest size variable.

- Changing the value of a member will not affect the other variable members
- The Size of the Structure is equal to the Sum of the Sizes of each data members

eg:

- only one variable member will be assessed at a time
- changing the value of a member will be affected the other variable members

eg:

Q) Illustrate the Purpose of return Statement using an example?

A)

Q) what are different Storage classes in C?
with one example?

A)

- Automatic Storage class → It is a default S.C

eg =

- External Storage class → It is a global variable

eg =

- Static Storage class → It is a local variable

eg =

- Register Storage class → It is a variable which is stored inside a Register.

eg =

Q) What is Recursion? Write a C Program?

A)

The process in which a function calls itself is known as recursion and the corresponding function is called recursive function.

```
#include <stdio.h>
int factorial(int n);
int main()
{
    int n;
    printf("Enter the number");
    scanf("%d", &n);
    int result = factorial(n);
    printf("%d", result);
    return 0;
}
```

```
int factorial(int n);
{
    if (n == 1)
    {
        return 1;
    }
    else
    {
        return n * factorial(n-1);
    }
}
```

Module 5

Q) explain the use of f seek() function?

A) Set the File Pointer to given position

Q) Distinguish b/w text mode and binary mode operations of a file?

- A)
- in text mode the Program writes data to file as text character.
 - in binary mode the Program writes data to files as 0/1 bits.
 - Text mode are organised around lines, each or which ends with a newline character.
 - binary mode is the one in which data is stored in the file in the same way as it is stored in the main memory for processing.

Q) what do you mean't by pointer variable ?
how it is initialized ?

A)

Pointer is a variable that can hold the address of a memory location. A pointer points to the address of memory location.

Pointer declaration :

Data Type * Pointer Variable;

Initialization of Pointer

Ptr = &a;

- here Pointer Variable. Ptr contains the address of variable a.
- The address of Operator (&) is used to retrieve the address of a variable.

Q) File handling operators

- 1) fopen() → open new file → fopen("file.txt", "w");
- 2) fprintf() → write data to file → fprintf(fp, "Hello file by", f, "n");
- 3) fscanf() → reads data from file → fscanf(fp, "%s", buff);
- 4) fclose() → closes the file → fclose(fp)
- 5) fgetc() → ~~write character to file~~ → fgetc(fp)
read a character from the file

Q) Diff b/w Address Operator & Indirection Operator ?

A) The address operator returns the address of the memory location in which the variable is stored.

• The indirection operator (*) gets the value stored in the memory location whose address is stored in a Pointer Variable.

Q) Diff b/w array of Pointers & Pointer to an array ?

Pointer to an array

- A user creates a Pointer for storing the address of any given array.

- It is alternatively known as an array pointer.

- dynamic in nature

array of Pointers

- A user creates an array of pointers that basically act as an array.

- Known as Pointer array

It is static in

nature

Q) Explain the different modes of file operations performed on file in C?

A)

Modes

r → opens a text file in read mode

w → opens a text file in write mode

a → opens a text file in append mode

r+ → opens a text file in read and write mode

w+ → opens a text file in read and write mode

a+ → opens a text file in read and write mode.